

시키지도 않은 시공비 청구서

같은 결과물, 다른 청구서

모델은 세 가지로 청구한다 — 입력 · 출력 · 사고

오늘은 그 청구서를 줄이는 습관 다섯 가지를 다룬다

숫자는 실측 · 로그 재계산 · 가정 시나리오를 구분해 말한다

오늘 가져갈 결론

비용은 모델이 아니라 습관이 만든다

영어로 생각하고 , 필요한 언어로 답한다

설명은 최소로 , 추론 강도는 작업에 맞춰

결정을 스펙으로 먼저 주고 , 반복되는 접두부는 캐싱한다

청구서의 구조

출력은 입력의 2~8 배

캐시된 입력과 비교하면 최대 50 배까지 벌어진다

사고 토큰은 출력 단가로 청구된다

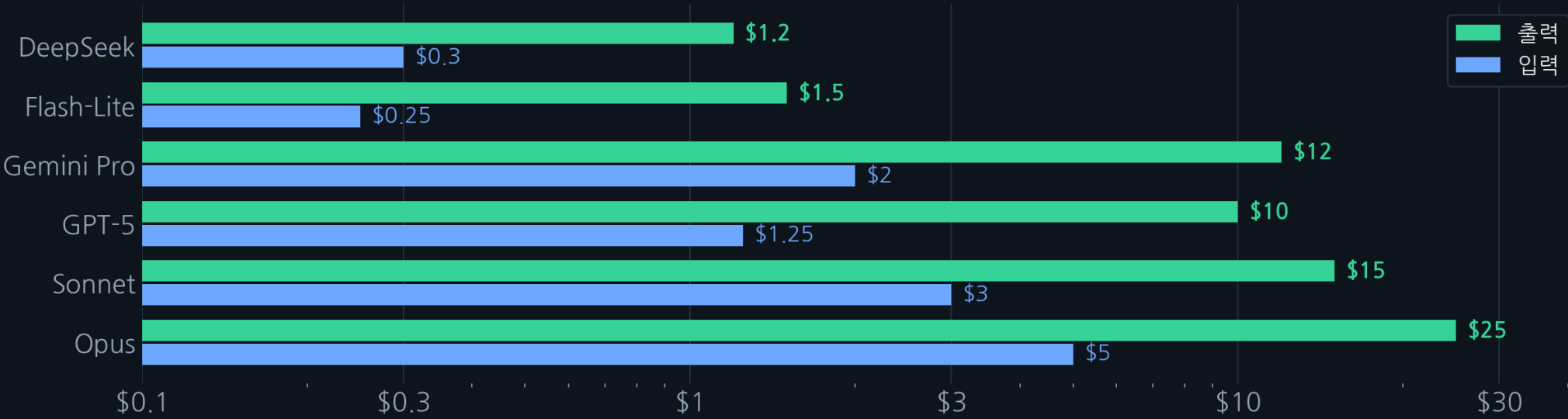
그래서 '덜 쓰기' 보다 '다시 안 보내기' 가 크다

같은 토큰 , 다른 단가

출력은 입력의 4~8 배

비싼 쪽 (출력) 을 줄이는 습관이 먼저다
캐시 읽기는 입력의 0.1 배 — 캐시가 있는 이유

모델별 입력·출력 단가 (USD / 100만 토큰, 로그 축)



단가 표로 다시 보기

출력 ÷ 입력 4~8 배 , 캐시 읽기 0.02~0.1 배

단가는 2026-09-20 공개 리스트 스냅샷 (USD / 100 만 토큰)

라이브 조회값이 아니라 저장된 표라서 , 갱신은 한 곳에서 한다

모델	입력 \$/1M	출력 \$/1M	출력 ÷ 입력	캐시 읽기
Claude Opus 4.6	\$5	\$25	5 배	0.1 배
Claude Sonnet 4.6	\$3	\$15	5 배	0.1 배
GPT-5	\$1.25	\$10	8 배	0.1 배
Gemini 3.1 Pro Preview (≤200k)	\$2	\$12	6 배	0.1 배
Gemini 3.1 Flash-Lite	\$0.25	\$1.5	6 배	0.1 배
DeepSeek V4.1 Flash (peak)	\$0.3	\$1.2	4 배	0.02 배

01 · 언어 습관

사고는 영어로 , 답은 사용자 언어로

토큰이 어디서 새는지 먼저 본다

원칙 1 · 영어로 생각하고, 필요한 언어로 답한다

같은 내용에 1.4723 배 · 구형 2.3669 배

FLORES-200 대응 번역 1,012 쌍을 전수 인코딩해 토큰 합계를 비교

짧고 압축된 문장일수록 격차가 커진다

사용자에게는 한국어로 답한다 — 안 보이는 사고만 영어로

왜 한국어가 더 쪼개지는가

토큰 / 글자 0.204 vs 0.526

어휘 20 만 개 중 한글은 약 1.2%

' 에이전트 ' 는 4 토큰 , ' agent ' 는 1 토큰

512 토큰이 영어 2,513 자 · 한국어 973 자 — 글자 수로 자르면 한국어 문서가 잘린다

한국어는 같은 내용에 토큰을 더 쓴다

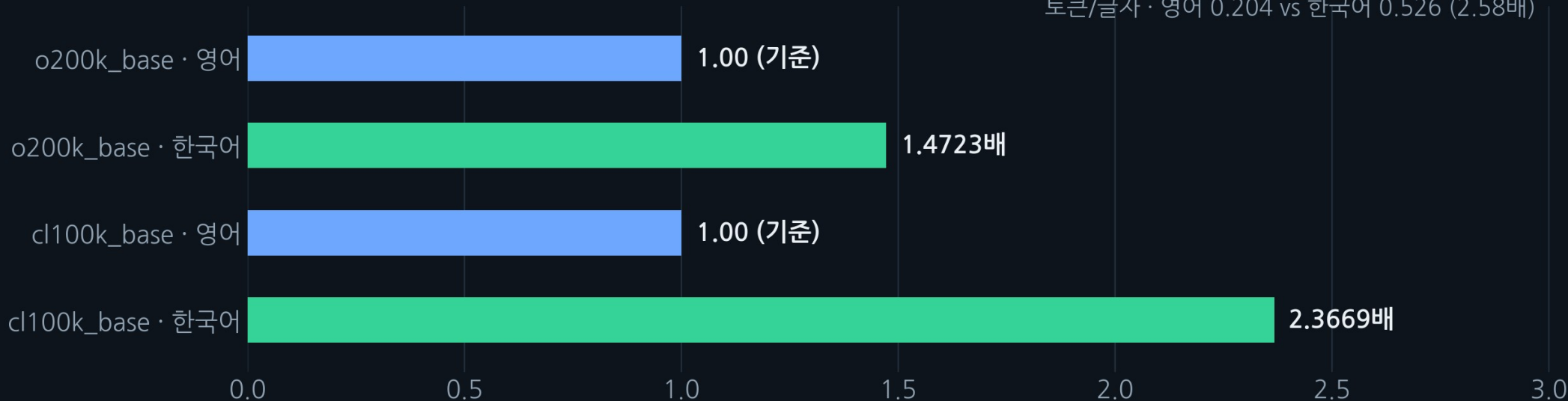
1.4723 배 · 2.3669 배

FLORES-200 대응 번역 1,012 쌍을 전수 인코딩

글자 수로 자르면 한국어가 먼저 잘린다

같은 내용(FLORES-200 1,012쌍)의 토큰 배수 — 영어 = 1.00

토큰/글자 · 영어 0.204 vs 한국어 0.526 (2.58배)



02 · 출력과 추론

비싼 쪽 (출력) 을 줄인다

설명 최소화 · 추론 강도 배분 · 스펙 선주입

원칙 2 · 설명을 최소화한다

342 → 58 토큰 (83% 감소)

월 1,760 건 기준 \$9.03 → \$1.53

' 코드를 읽으면 아는 문장 ' 만 지운다 — 코드는 그대로

샘플은 차이를 보이려고 극단적으로 잡았다 . 자기 작업으로 다시 재라

원칙 3 · 추론 강도를 작업에 맞춘다

월 \$924 → \$118 (70% 절감)

minimal \$7.92 · medium \$132 · max \$924

설계 10% 만 high, 정형 · 단순 작업은 low

reasoning/output 비율이 0.8 을 넘으면 과다 신호

원칙 4 · 결정을 스펙으로 먼저 준다

재작업 5 → 2 사이클, 68.7% 절감

스펙 4,000 토큰 작성 비용도 포함한 계산

얇은 스펙만으로도 36.9% 절감

설명이 아니라 결정을 적는다 — 페이지 · 스택 · 디자인 · 수용 기준

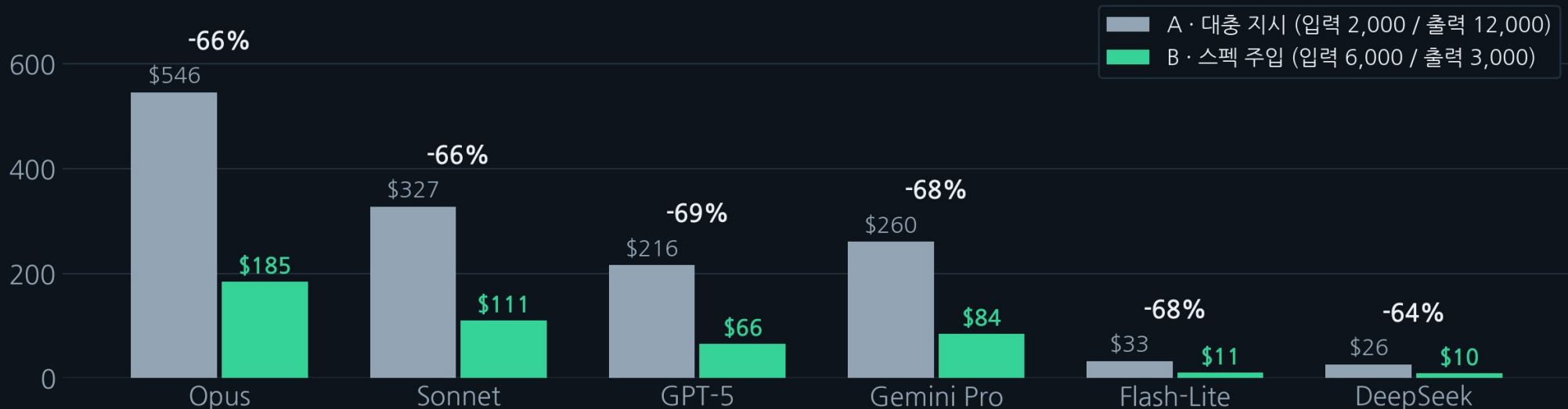
월 청구서로 보면

-66% (월 1,760 건)

같은 업무량 , 두 워크플로 — 입력 2,000/ 출력 12,000 vs 입력 6,000/ 출력 3,000

단가가 다른 모델에서도 방향은 같다

모델별 월 청구서 — 같은 1,760건, 두 워크플로 (USD)



원칙 5 · 턴 수가 비용을 지배한다

턴 2 배 → 비용 3.1 배

5 턴 \$0.26 · 20 턴 \$2.32 · 50 턴 \$12.09

안 쓰는 코드도 매 턴 다시 실행 과금된다 (YAGNI)

컨텍스트는 길이보다 '반복' 이 비쌉니다

긴 세션일수록 가파르다

5 턴 \$0.26 → 50 턴 \$12.09

20 턴에 컴팩션을 걸면 \$0.93 (60% 절감)

안 쓰는 코드도 매 턴 다시 실행 과금된다

턴 수에 따른 세션 비용 (Claude Sonnet 4.6, 고정 8,000토큰 프리픽스)



03 · 캐시와 컨텍스트

반복되는 것은 한 번만 낸다

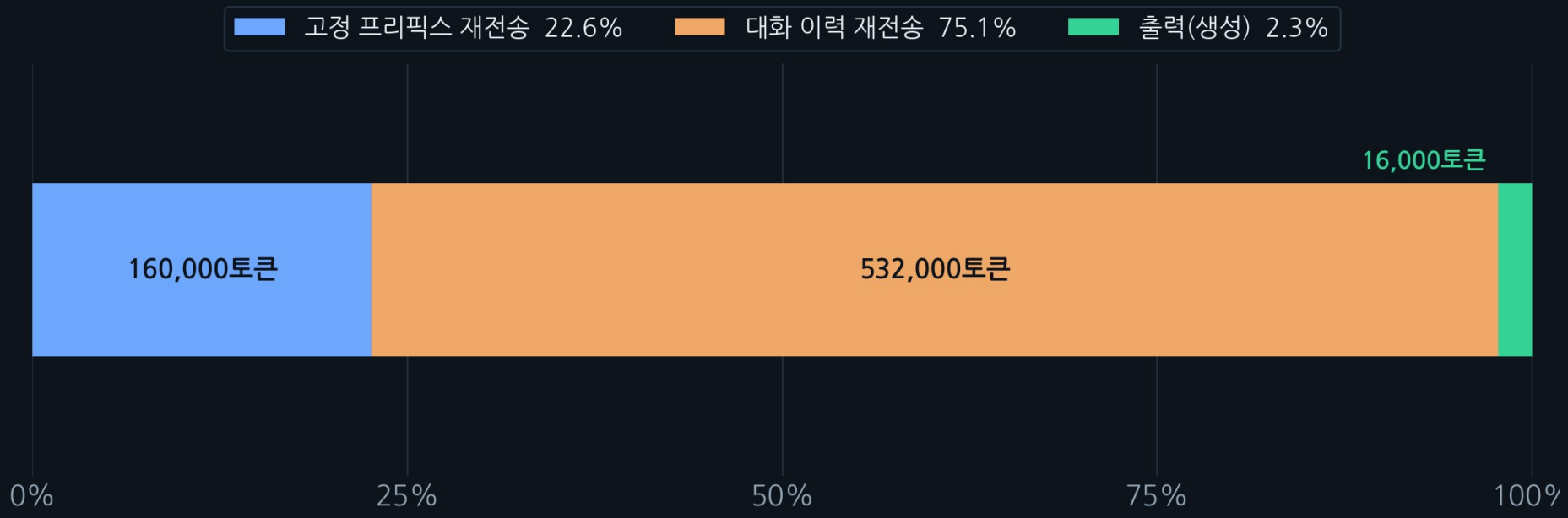
적중률을 올리고, 이력은 접고, 톨 출력은 줄인다

세션 토큰의 4분의 3은 이력 재전송

고정 프리픽스 22.6% · 이력 75.1%

생성 (출력) 은 2.3% 뿐이다

그래서 ' 덜 쓰기 ' 보다 ' 다시 안 보내기 ' 가 크다



캐싱 · 오늘 바로 켤 수 있는 레버

\$8.55 → \$3.22 (62.4% 절감)

쓰기 프리미엄은 두 번째 호출에서 회수된다

고정 블록 (시스템 · 톨 정의 · 규약) 을 맨 앞으로 모은다

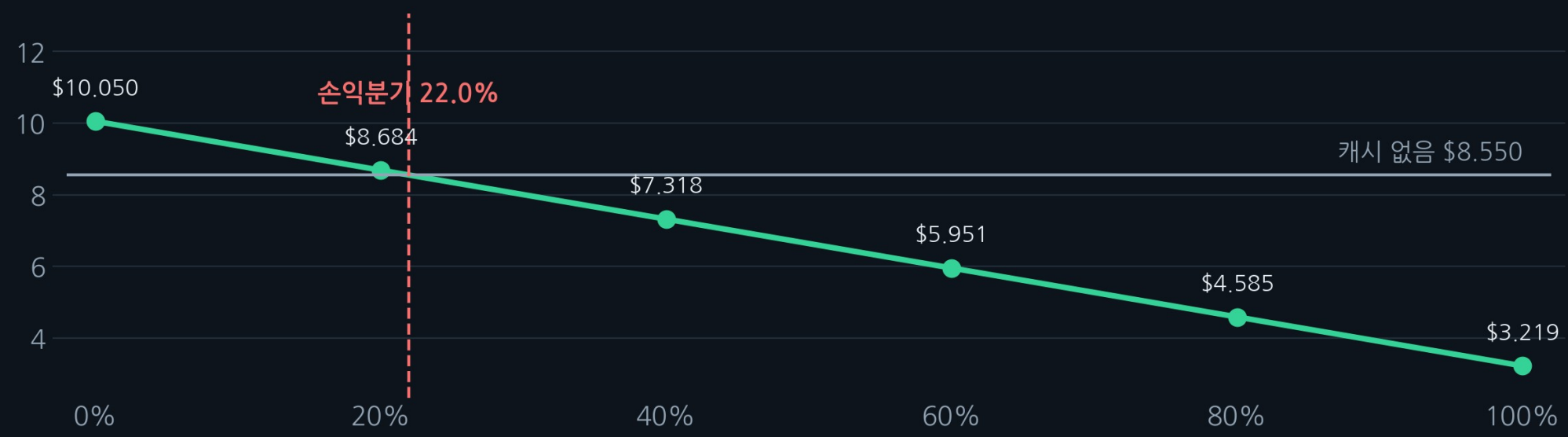
최소 길이를 확인한다 — Haiku 4.5 는 4,096 토큰

적중률 22% 면 본전

캐시 없음 \$8.550

적중률이 오를수록 단조 감소한다
미스가 섞이면 쓰기 프리미엄을 계속 문다

프롬프트 캐시 적중률에 따른 100콜 비용 (Sonnet 4.6)



캐시를 깨는 한 줄

\$10.05 (17.5% 더 지출)

맨 앞의 타임스탬프 · 요청 ID 는 매번 미스로 만든다

변하는 값은 반드시 뒤로 보낸다

미스 상태의 캐싱은 안 쓰느니만 못하다

캐시는 조건이 맞을 때만 이득

\$8.55 → \$3.219 (-62%)

프리픽스를 매번 바꾸면 \$10.05 (+17.5%)

고정 블록을 앞으로 모아 적중률을 올린다

같은 100콜, 캐시 조건만 바꿈 (Sonnet 4.6 단가)



압축과 캐싱은 대체재가 아니다

\$0.4014 → \$0.2007 (94% 절감)

그냥 캐싱 \$0.4014 vs 절반 요약 \$1.8000

지울 수 있는 반복은 지우고, 지울 수 없는 반복은 고정해 캐시에 태운다

압축률이 높을수록 조합의 이득도 커진다

긴 세션은 컴팩션으로 접는다

\$2.32 → \$0.93 (60% 절감)

10 턴마다 접으면 \$1.23 (47% 절감)

너무 자주 접으면 요약 호출이 이득을 먹는다

최적 주기는 세션 길이와 요약 비용으로 정해진다

툴 출력과 서버에이전트는 전체 비용으로 본다

\$5.6010 → \$2.7360

툴 읽기 누적 전송 1,680,000 토큰 (20 턴)

컴팩션 \$3.4425 · 서버에이전트 \$2.736

자식 호출 · 요약 비용까지 포함해 비교한다

모델 라우팅 · 쉬운 요청에 비싼 모델을 쓰지 않는다

전부 강 \$350 → 절반 \$184

절반만 라우팅해도 47% 절감

공격적 라우팅은 \$35.17 (90% 절감)

품질 측정 도구가 먼저다 — 임계값은 주기적으로 재보정

배치 API · 지연을 할인으로 바꾼다

50,000 건 \$675 → \$337.50

입력 · 출력 모두 50% 할인

24 시간 안에 끝나면 되는 잡만

야간 평가 · 대량 분류 · 리포트 생성 · 레포 전체 분석

두 습관, 같은 결과물

페르소나 19.0 배 (기존 가정 20.8 배)

A: 6 턴 · 전체 재출력 · 과한 사고 / B: 2 턴 · 짧은 패치

연 환산 \$18,943 차이 (기존 가정 \$20,832)

같은 작업 1,760 건 / 년을 가정한 비교

작은 모델이 지휘한다

사다리 2.81 배 (1 턴 27% · 2 턴 43%)

실패가 반복되면 격차가 커진다 — 6 회 2.80 배

짧은 리워크에서도 이득 — 1 턴 27% · 2 턴 43%

respec 이면 실행 전에 멈추므로 실행 비용만 계산

다섯 레버를 겹치면

잔여 14.18%

시맨틱 캐시 → 라우팅 → 캐싱 → effort → 배치

각 레버가 자기 조건을 만족할 때의 순차 산술

레버를 순서대로 적용한 잔여 비용 (%)



사고 예산을 적지 않으면 기본값이 정한다

미지정 대비 199.47 배

미지정 \$0.000028 vs 자동 \$0.00558525 (호출당)

예산을 명시하는 순간 청구서가 예측 가능해진다

사고 예산별 호출당 비용 (저장 로그 재계산, 로그 축)



안 보이는 사고 토큰

199.47 배 (기존 환산 240.02 배)

같은 문제를 조건당 6 회 실행한 저장 로그 30 행

사고가 난 호출에서 사고 토큰 비중 99.625%

예산을 지정하지 않으면 사실상 ' 무제한 ' — 명시적으로 정한다

언어별 비용은 지표마다 다르다

호출당 1.364 배 · 0.729 배

문자당 (기존 환산 단가) 3.027 배 · 1.363 배

같은 과제라도 언어에 따라 생성 길이가 달라진다

지표를 섞으면 결론이 뒤집힌다

04 · 사례와 적용

실제 한 페이지에서 확인한다

같은 목표, 두 가지 과정 — 그리고 팀에 붙이는 순서

사례 · 카페 랜딩 한 페이지

토큰 +1.8% · 비용 8.3% 감소

한 문장 지시 vs 4 필드 스펙 — 같은 목표

GPT-5 8.3% · Sonnet 7.6% · 캐시 미적용 8.1%

입력은 늘고, 비싼 출력은 줄었다

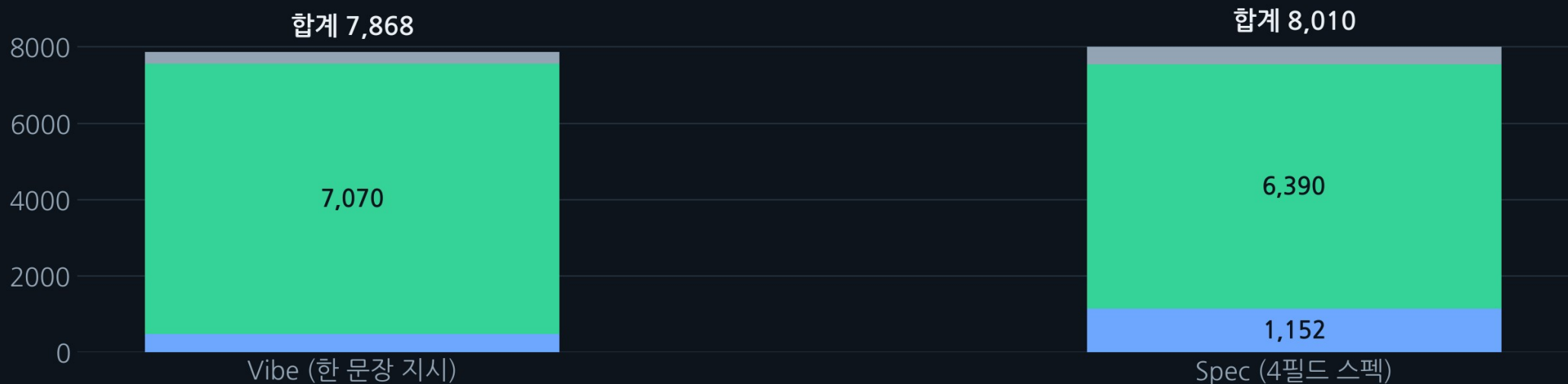
무엇이 비용을 움직였나

합계 7,868 → 8,010 (+1.8%)

비캐시 입력 ↑ · 추론 + 출력 ↓

합계는 거의 그대로, 구조만 바뀌었다

두 산출물의 토큰 구조 (추정 usage)



사례 · 무엇을 확인했나

정적 16 + 브라우저 29 항목

390 · 820 · 821 · 1280px 실제 렌더링 검사

카드 내용 · 앵커 이동 · 가로 스크롤 · JS 오류 0 건

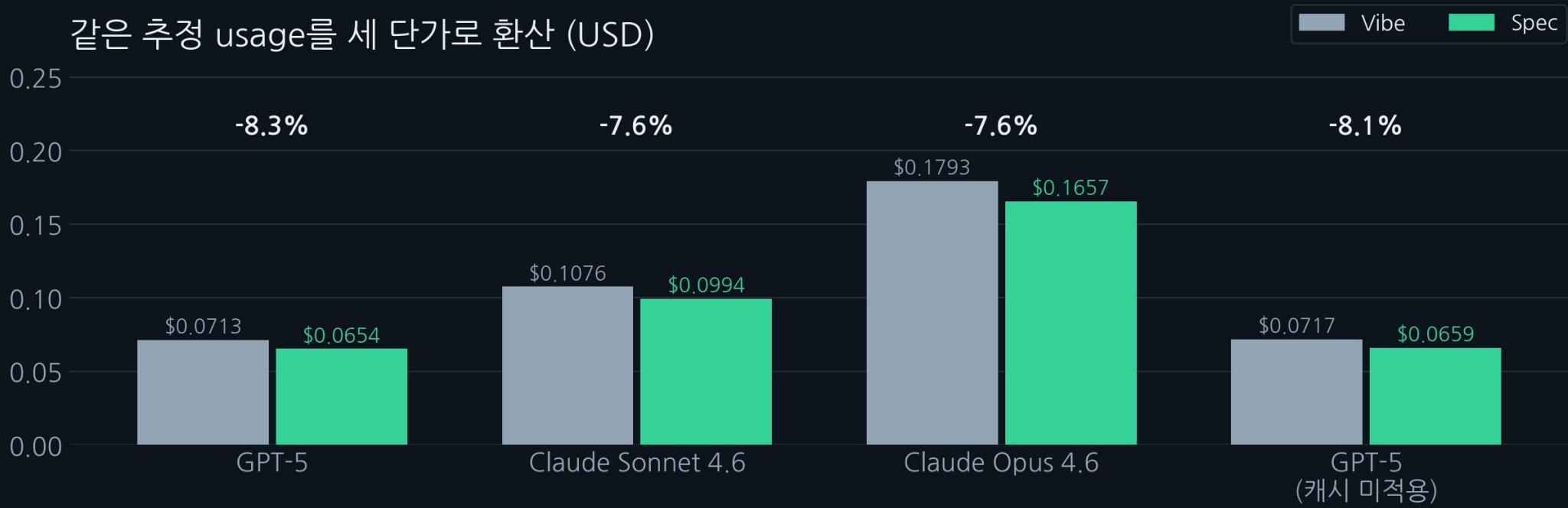
통과 여부는 실행 로그에 남는다

단가가 바뀌어도 방향은 같다

감소 8.3% · 7.6% · 8.1%

같은 추정 usage 를 세 단가로 환산

출력 단가가 높을수록 격차가 커진다



적용 우선순위

1~3 번은 오늘 오후에 켤 수 있다

낮은 난이도 : 캐싱 · verbosity/effort · 배치

중간 : 스펙 먼저 · 컴팩션

높은 난이도 : 라우팅 · 시맨틱 캐시 · 압축

팀에 붙이는 순서

측정 → 적용 → 재측정

usage 로그로 자기 배수를 먼저 켜다

고정 접두부를 앞으로, 변하는 값은 뒤로

캐시 적중률과 reasoning/output 비율을 대시보드로 본다

흔한 실수 다섯

미스 캐싱 · 과잉 추론 · 전체 재출력 · 오배치 라우팅 · 무제한 사고

캐시를 켜놓고 프리픽스를 매일 바꾼다

'전체 파일 다시 써줘'로 출력을 다시 산다

라우팅 임계값을 벤치마크 값 그대로 쓴다

캐싱과 개인정보

개인화 · 실시간 응답은 캐싱 금지

멀티턴은 캐시 키에 컨텍스트를 포함

시맨틱 캐시 임계값이 느슨하면 '비슷하지만 틀린 답'을 준다

삭제 · TTL 정책을 프롬프트 설계와 함께 정한다

직접 확인하는 방법

python run_all.py (유료 호출 0)

실험 11 종이 오프라인으로 전부 재현된다

원자료 · 로그 · 검증 스크립트 · 발표 덱 생성기를 함께 공개

덱과 대본도 같은 원천에서 생성된다

작게 시작해서 자기 usage 로 확인한다

오늘 한 가지만 켜라 — 캐시

다음 주에 배수와 적중률을 다시 켜다

품질 게이트를 함께 둔다 — 절감만으로 대신하지 않는다

질문과 재현 결과는 저장소 이슈로 남긴다